Practical 1: Business Data Collection and Exploratory Data Analysis (EDA)

Aim: Collect sample business data and use summary statistics and charts to identify patterns.

Program:

```
# Practical 1: Collect business data and perform EDA
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Example business dataset: monthly sales by product
df = pd.DataFrame({
    "Month": ["Jan", "Feb", "Mar", "Apr", "May", "Jun"],
    "Product": ["A", "A", "B", "A", "B", "B"],
    "Sales": [12000, 15000, 11000, 18000, 16000, 21000],
    "Units_Sold": [120, 145, 105, 170, 155, 200],
    "Advertising": [1500, 1800, 1200, 2200, 2000, 2600]
})

print("BUSINESS DATA")
print(df)
print("\nFirst five rows:\n", df.head())
print("\nDataset information:")
df.info()
print("\nMissing values:\n", df.isnull().sum())
print("\nDescriptive statistics:\n", df.describe())
print("\nAverage sales:", df["Sales"].mean())
print("Total sales:", df["Sales"].sum())

sns.histplot(data=df, x="Sales", kde=True)
plt.title("Distribution of Monthly Sales")
plt.xlabel("Sales (Rs.)")
plt.tight_layout()
plt.show()

sns.scatterplot(data=df, x="Advertising", y="Sales", hue="Product")
plt.title("Advertising vs Sales")
plt.tight_layout()
plt.show()

print("\nResult: EDA was performed to summarize sales, check missing values, and visualize patterns.")
```

Result / Interpretation: The program performs the stated analysis and prints the calculated values. Interpret these values in the context of the sample business data; the data is illustrative, not evidence about a real company.

Practical 2: Central Limit Theorem

Aim: Study the sampling distribution of the sample mean using repeated random samples.

Program:

```
# Practical 2: Central Limit Theorem
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)
population = np.random.exponential(scale=50, size=10000)
sample_size = 30
number_of_samples = 1000

sample_means = [
    np.mean(np.random.choice(population, size=sample_size, replace=True))
    for _ in range(number_of_samples)
]

print("Population mean:", round(np.mean(population), 2))
print("Mean of sample means:", round(np.mean(sample_means), 2))
print("Standard deviation of sample means:", round(np.std(sample_means, ddof=1), 2))

plt.hist(population, bins=40, density=True, alpha=0.7)
plt.title("Original Population Distribution")
plt.xlabel("Value")
plt.ylabel("Density")
plt.tight_layout()
plt.show()

plt.hist(sample_means, bins=35, density=True, alpha=0.7)
plt.title("Sampling Distribution of Sample Means (n=30)")
plt.xlabel("Sample mean")
plt.ylabel("Density")
plt.tight_layout()
plt.show()

print("\nResult: The distribution of sample means is approximately normal and centered near the population mean.")
```

Result / Interpretation: The program performs the stated analysis and prints the calculated values. Interpret these values in the context of the sample business data; the data is illustrative, not evidence about a real company.

Practical 3: Descriptive Statistics Using Sales.csv

Aim: Calculate mean, median, mode, range, variance, standard deviation, and quartiles.

Program:

```
# Practical 3: Descriptive statistics using Sales.csv
import os
import pandas as pd
from scipy import stats

filename = "Sales.csv"

# Create an example file if Sales.csv is not in this folder.
if not os.path.exists(filename):
    example = pd.DataFrame({
        "Sales": [12000, 15000, 11000, 18000, 16000, 21000, 17500, 19000, 14000, 22000]
    })
    example.to_csv(filename, index=False)
    print("Sample Sales.csv created because no file was found.")

df = pd.read_csv(filename)
print("Columns available:", list(df.columns))
print("\nFirst five rows:\n", df.head())

column = "Sales" if "Sales" in df.columns else df.select_dtypes(include="number").columns[0]
data = pd.to_numeric(df[column], errors="coerce").dropna()

print(f"\nStatistics for '{column}':")
print("Count:", data.count())
print("Mean:", round(data.mean(), 2))
print("Median:", round(data.median(), 2))
mode_values = stats.mode(data, keepdims=True).mode
print("Mode:", mode_values.tolist())
print("Minimum:", data.min())
print("Maximum:", data.max())
print("Range:", data.max() - data.min())
print("Variance (sample):", round(data.var(ddof=1), 2))
print("Standard deviation (sample):", round(data.std(ddof=1), 2))
print("Q1:", data.quantile(0.25))
print("Q3:", data.quantile(0.75))

print("\nResult: Descriptive statistics were calculated for the selected sales variable.")
```

Result / Interpretation: The program performs the stated analysis and prints the calculated values. Interpret these values in the context of the sample business data; the data is illustrative, not evidence about a real company.

Practical 4: Confidence Interval

Aim: Estimate a 95% confidence interval for the population mean of weekly sales.

Program:

```
# Practical 4: Confidence interval for a population mean
import numpy as np
from scipy import stats

sales = np.array([52, 55, 53, 57, 54, 56, 51, 58, 55, 54,
                  53, 59, 56, 55, 57, 54, 52, 56, 58, 55], dtype=float)
# Sales are in thousands of rupees.
confidence_level = 0.95
n = len(sales)
mean = np.mean(sales)
standard_error = stats.sem(sales)
lower, upper = stats.t.interval(confidence_level, df=n - 1,
                                loc=mean, scale=standard_error)

print("Sample mean sales (Rs. thousands):", round(mean, 2))
print("95% confidence interval (Rs. thousands):",
      (round(lower, 2), round(upper, 2)))
print("In rupees: Rs.", round(lower * 1000, 2), "to Rs.", round(upper * 1000, 2))
print("\nInterpretation: The interval gives a plausible range for the population mean weekly sales.")
print("Result: A 95% confidence interval was calculated using the t-distribution.")
```

Result / Interpretation: The program performs the stated analysis and prints the calculated values. Interpret these values in the context of the sample business data; the data is illustrative, not evidence about a real company.

Practical 5: Data Visualization Using Histogram and Box Plot

Aim: Visualize sales distribution and identify spread and possible outliers.

Program:

```
# Practical 5: Histogram and box plot
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.DataFrame({"Sales": [12000, 15000, 11000, 18000, 16000, 21000,
                             17500, 19000, 14000, 22000, 13000, 20000]})

print(df["Sales"].describe())

plt.figure()
sns.histplot(data=df, x="Sales", bins=6, kde=True)
plt.title("Histogram of Sales")
plt.xlabel("Sales (Rs.)")
plt.tight_layout()
plt.show()

plt.figure()
sns.boxplot(data=df, x="Sales")
plt.title("Box Plot of Sales")
plt.xlabel("Sales (Rs.)")
plt.tight_layout()
plt.show()

print("\nInterpretation: The histogram shows the distribution of sales; the box plot shows the median, spread, and possible outliers.")
print("Result: Sales distribution was visualized using a histogram and box plot.")
```

Result / Interpretation: The program performs the stated analysis and prints the calculated values. Interpret these values in the context of the sample business data; the data is illustrative, not evidence about a real company.

Practical 6: Bootstrap Confidence Interval

Aim: Use bootstrap resampling to estimate a confidence interval for median sales.

Program:

```
# Practical 6: Bootstrap confidence interval for median sales
import numpy as np

sales = np.array([12000, 15000, 11000, 18000, 16000, 21000,
                  17500, 19000, 14000, 22000, 13000, 20000])
np.random.seed(42)

bootstrap_medians = []
iterations = 10000
for _ in range(iterations):
    sample = np.random.choice(sales, size=len(sales), replace=True)
    bootstrap_medians.append(np.median(sample))

lower, upper = np.percentile(bootstrap_medians, [2.5, 97.5])

print("Original median sales:", np.median(sales))
print("Bootstrap 95% confidence interval for median:",
      (round(lower, 2), round(upper, 2)))
print("\nInterpretation: The interval estimates uncertainty in the population median using repeated resampling.")
print("Result: A bootstrap confidence interval was calculated for median sales.")
```

Result / Interpretation: The program performs the stated analysis and prints the calculated values. Interpret these values in the context of the sample business data; the data is illustrative, not evidence about a real company.

Practical 7: Correlation Analysis

Aim: Measure the strength and direction of relationships among business variables.

Program:

```
# Practical 7: Correlation analysis between advertising and sales
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from scipy.stats import pearsonr

df = pd.DataFrame({
    "Advertising": [1000, 1200, 1500, 1700, 2000, 2200, 2500, 2800],
    "Sales": [10000, 12000, 13000, 15000, 17000, 18000, 21000, 23000],
    "Customer_Rating": [3.2, 3.4, 3.6, 3.7, 4.0, 4.1, 4.3, 4.5]
})

print(df)
corr, p_value = pearsonr(df["Advertising"], df["Sales"])
print("\nPearson correlation (advertising vs sales):", round(corr, 4))
print("P-value:", round(p_value, 6))

print("\nCorrelation matrix:\n", df.corr(numeric_only=True))

sns.heatmap(df.corr(numeric_only=True), annot=True, cmap="coolwarm", vmin=-1, vmax=1)
plt.title("Business Variable Correlation Matrix")
plt.tight_layout()
plt.show()

if corr > 0:
    print("Interpretation: Advertising and sales have a positive linear relationship in this sample.")
elif corr < 0:
    print("Interpretation: Advertising and sales have a negative linear relationship in this sample.")
else:
    print("Interpretation: No linear correlation was found in this sample.")

print("Result: Correlation analysis was performed. Correlation does not by itself prove causation.")
```

Result / Interpretation: The program performs the stated analysis and prints the calculated values. Interpret these values in the context of the sample business data; the data is illustrative, not evidence about a real company.

Practical 8: Hypothesis Testing

Aim: Test whether average weekly sales are greater than a reference value of Rs. 50,000.

Program:

```
# Practical 8: Hypothesis testing for a business problem
import numpy as np
from scipy import stats

# Weekly sales in thousands of rupees after a campaign (illustrative data)
sales = np.array([52, 55, 53, 57, 54, 56, 51, 58, 55, 54,
                  53, 59, 56, 55, 57, 54, 52, 56, 58, 55], dtype=float)

# H0: Population mean sales = 50 (Rs. 50,000)
# H1: Population mean sales > 50 (Rs. 50,000)
population_mean = 50
alpha = 0.05

t_stat, p_two_tailed = stats.ttest_1samp(sales, population_mean)
p_one_tailed = p_two_tailed / 2 if t_stat > 0 else 1 - p_two_tailed / 2

print("HYPOTHESIS TESTING - BUSINESS SALES")
print("-----------------------------------")
print("H0: Mean weekly sales = Rs. 50,000")
print("H1: Mean weekly sales > Rs. 50,000")
print(f"Sample mean sales: Rs. {np.mean(sales) * 1000:.2f}")
print(f"t-statistic: {t_stat:.4f}")
print(f"One-tailed p-value: {p_one_tailed:.8f}")
print(f"Significance level: {alpha}")

if p_one_tailed < alpha:
    print("Decision: Reject H0.")
    print("Interpretation: There is statistically significant evidence that mean sales exceed Rs. 50,000.")
else:
    print("Decision: Fail to reject H0.")
    print("Interpretation: There is insufficient evidence that mean sales exceed Rs. 50,000.")

print("Result: A one-sample, one-tailed t-test was performed.")
```

Result / Interpretation: The program performs the stated analysis and prints the calculated values. Interpret these values in the context of the sample business data; the data is illustrative, not evidence about a real company.

Practical 9: Probability Experiment Using Dice

Aim: Simulate die rolls and compare experimental probabilities with theoretical probabilities.

Program:

```
# Practical 9: Simulate rolling a fair die
import numpy as np
from collections import Counter
import matplotlib.pyplot as plt

np.random.seed(42)
number_of_rolls = 10000
rolls = np.random.randint(1, 7, size=number_of_rolls)
counts = Counter(rolls)

print("Number of rolls:", number_of_rolls)
for face in range(1, 7):
    experimental_probability = counts[face] / number_of_rolls
    print(f"Face {face}: count={counts[face]}, experimental probability={experimental_probability:.4f}")

print("Theoretical probability of each face:", 1 / 6)

plt.bar(range(1, 7), [counts[i] / number_of_rolls for i in range(1, 7)])
plt.axhline(1/6, linestyle="--", label="Theoretical probability")
plt.xlabel("Die face")
plt.ylabel("Probability")
plt.title("Experimental vs Theoretical Dice Probabilities")
plt.legend()
plt.tight_layout()
plt.show()

print("\nResult: A die-rolling experiment was simulated and experimental probabilities were compared with theoretical probabilities.")
```

Result / Interpretation: The program performs the stated analysis and prints the calculated values. Interpret these values in the context of the sample business data; the data is illustrative, not evidence about a real company.

Practical 10: Random Probability Distributions

Aim: Generate and analyze normal, uniform, and exponential random samples.

Program:

```
# Practical 10: Generate and analyze probability distributions
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)
size = 5000

normal_data = np.random.normal(loc=50, scale=10, size=size)
uniform_data = np.random.uniform(low=0, high=100, size=size)
exponential_data = np.random.exponential(scale=20, size=size)

datasets = [
    ("Normal distribution", normal_data),
    ("Uniform distribution", uniform_data),
    ("Exponential distribution", exponential_data)
]

for name, data in datasets:
    print(f"\n{name}")
    print("Mean:", round(np.mean(data), 2))
    print("Standard deviation:", round(np.std(data, ddof=1), 2))
    print("Minimum:", round(np.min(data), 2))
    print("Maximum:", round(np.max(data), 2))
    plt.figure()
    plt.hist(data, bins=40, density=True, alpha=0.75)
    plt.title(name)
    plt.xlabel("Value")
    plt.ylabel("Density")
    plt.tight_layout()
    plt.show()

print("\nResult: Random samples were generated from normal, uniform, and exponential distributions and their properties were analyzed."
```

Result / Interpretation: The program performs the stated analysis and prints the calculated values. Interpret these values in the context of the sample business data; the data is illustrative, not evidence about a real company.

Practical 11: Decision Tree Model

Aim: Train a decision tree classifier using example customer purchase data.

Program:

```
# Practical 11: Decision tree for business decisions
import pandas as pd
from sklearn.tree import DecisionTreeClassifier, plot_tree
import matplotlib.pyplot as plt

# Example: predict whether a customer will purchase based on visit and discount
df = pd.DataFrame({
    "Website_Visits": [1, 2, 3, 4, 5, 6, 2, 7, 8, 3, 6, 9],
    "Discount_Percent": [0, 5, 10, 10, 15, 20, 0, 20, 25, 5, 15, 25],
    "Purchased": [0, 0, 0, 1, 1, 1, 0, 1, 1, 0, 1, 1]
})

X = df[["Website_Visits", "Discount_Percent"]]
y = df["Purchased"]

model = DecisionTreeClassifier(max_depth=3, random_state=42)
model.fit(X, y)

new_customer = pd.DataFrame({
    "Website_Visits": [5],
    "Discount_Percent": [10]
})
prediction = model.predict(new_customer)[0]
print("Training data:\n", df)
print("\nPredicted purchase (1=yes, 0=no):", int(prediction))
print("Prediction label:", "Likely to purchase" if prediction == 1 else "Unlikely to purchase")

plt.figure(figsize=(12, 7))
plot_tree(model, feature_names=X.columns, class_names=["No", "Yes"],
          filled=True, rounded=True)
plt.title("Decision Tree for Customer Purchase")
plt.tight_layout()
plt.show()

print("\nResult: A decision tree classifier was trained to predict customer purchase from example business data.")
```

Result / Interpretation: The program performs the stated analysis and prints the calculated values. Interpret these values in the context of the sample business data; the data is illustrative, not evidence about a real company.

Practical 12: Expected Value for Business Decisions

Aim: Compare business alternatives using expected profit.

Program:

```
# Practical 12: Expected value for business decisions
# A business is choosing between two marketing campaigns.
# Profit outcomes are illustrative and expressed in rupees.

import numpy as np

# Campaign A: high certainty, moderate return
profits_A = np.array([50000, 50000, 50000])
probabilities_A = np.array([0.3, 0.4, 0.3])

# Campaign B: uncertain return
profits_B = np.array([20000, 60000, 100000])
probabilities_B = np.array([0.3, 0.4, 0.3])

expected_A = np.sum(profits_A * probabilities_A)
expected_B = np.sum(profits_B * probabilities_B)

print("CAMPAIGN A")
for profit, probability in zip(profits_A, probabilities_A):
    print(f"Profit Rs. {profit}: probability {probability}")
print("Expected value of Campaign A: Rs.", round(expected_A, 2))

print("\nCAMPAIGN B")
for profit, probability in zip(profits_B, probabilities_B):
    print(f"Profit Rs. {profit}: probability {probability}")
print("Expected value of Campaign B: Rs.", round(expected_B, 2))

if expected_A > expected_B:
    print("\nDecision: Choose Campaign A based on higher expected profit.")
elif expected_B > expected_A:
    print("\nDecision: Choose Campaign B based on higher expected profit.")
else:
    print("\nDecision: Both campaigns have the same expected profit.")

print("\nResult: Expected values were calculated and used to compare business alternatives.")
```

Result / Interpretation: The program performs the stated analysis and prints the calculated values. Interpret these values in the context of the sample business data; the data is illustrative, not evidence about a real company.